

COMP3242/6242: Deep Learning

Week 4 — Image Classification, Object Detection and Convolutional Neural Networks

Stephen Gould

`stephen.gould@anu.edu.au`

Australian National University

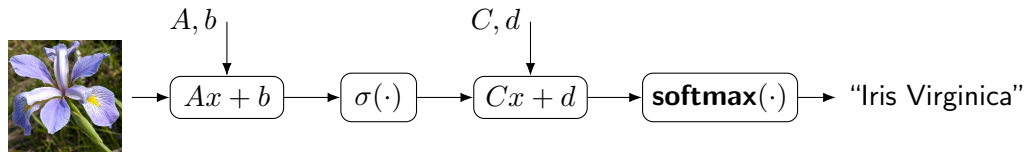
Semester 1, 2026

Overview

- ▶ why not multi-layer perceptrons for images?
- ▶ what are convolutions (1D, 2D and 3D)?
- ▶ back-propagating through convolutions
- ▶ pooling operations
- ▶ convolutional neural networks (CNNs)
- ▶ putting it all together for image classification
- ▶ evaluation

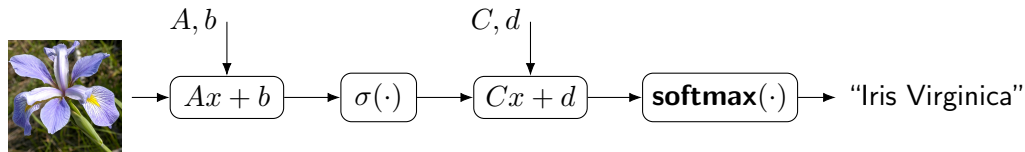
Hypothetical Image Classifier

- ▶ consider using a multi-layer perceptron for image classifier



Hypothetical Image Classifier

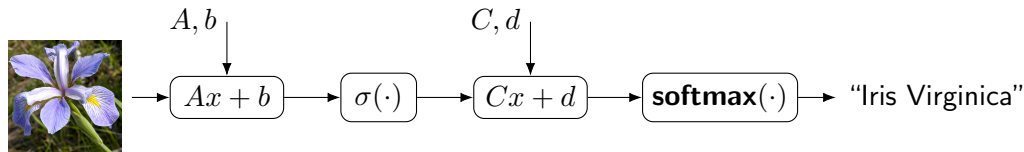
- ▶ consider using a multi-layer perceptron for image classifier



- ▶ let the image be 224-by-224 pixels

Hypothetical Image Classifier

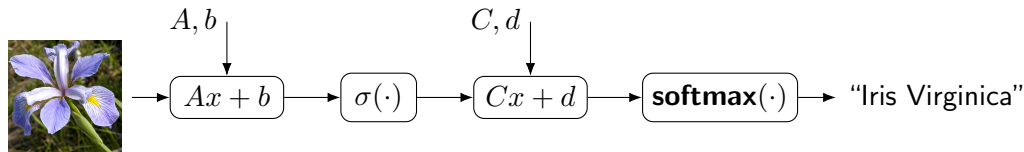
- ▶ consider using a multi-layer perceptron for image classifier



- ▶ let the image be 224-by-224 pixels
- ▶ that's $n = 224 \times 224 \times 3 = 150,528$ inputs (vectorized)

Hypothetical Image Classifier

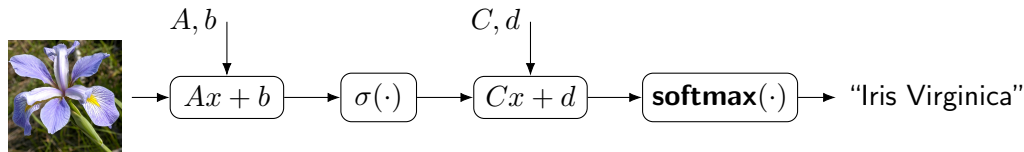
- ▶ consider using a multi-layer perceptron for image classifier



- ▶ let the image be 224-by-224 pixels
- ▶ that's $n = 224 \times 224 \times 3 = 150,528$ inputs (vectorized)
- ▶ let's have a 1000-dimensional hidden state and 10 output categories

Hypothetical Image Classifier

- ▶ consider using a multi-layer perceptron for image classifier

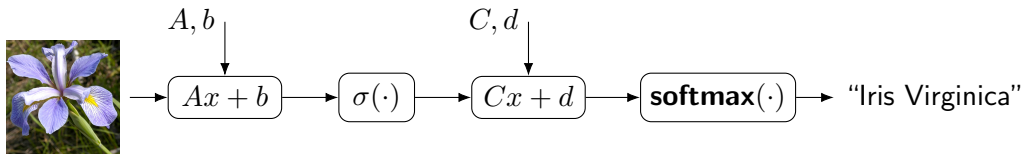


- ▶ let the image be 224-by-224 pixels
- ▶ that's $n = 224 \times 224 \times 3 = 150,528$ inputs (vectorized)
- ▶ let's have a 1000-dimensional hidden state and 10 output categories
- ▶ therefore

$$A \in \mathbb{R}^{150528 \times 1000}, \quad b \in \mathbb{R}^{1000}, \quad C \in \mathbb{R}^{1000 \times 10}, \quad d \in \mathbb{R}^{10}$$

Hypothetical Image Classifier

- ▶ consider using a multi-layer perceptron for image classifier



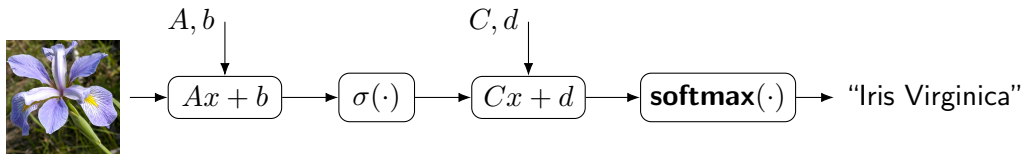
- ▶ let the image be 224-by-224 pixels
- ▶ that's $n = 224 \times 224 \times 3 = 150,528$ inputs (vectorized)
- ▶ let's have a 1000-dimensional hidden state and 10 output categories
- ▶ therefore

$$A \in \mathbb{R}^{150528 \times 1000}, \quad b \in \mathbb{R}^{1000}, \quad C \in \mathbb{R}^{1000 \times 10}, \quad d \in \mathbb{R}^{10}$$

- ▶ assume 32-bit floating-point parameters (i.e., 4 bytes per parameter)

Hypothetical Image Classifier

- ▶ consider using a multi-layer perceptron for image classifier



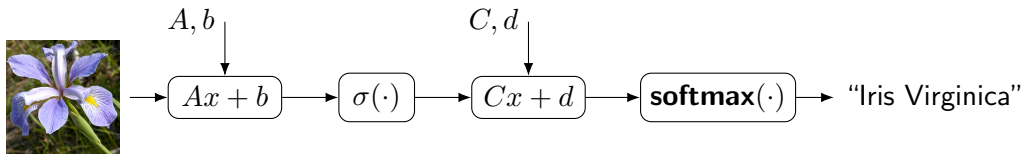
- ▶ let the image be 224-by-224 pixels
- ▶ that's $n = 224 \times 224 \times 3 = 150,528$ inputs (vectorized)
- ▶ let's have a 1000-dimensional hidden state and 10 output categories
- ▶ therefore

$$A \in \mathbb{R}^{150528 \times 1000}, \quad b \in \mathbb{R}^{1000}, \quad C \in \mathbb{R}^{1000 \times 10}, \quad d \in \mathbb{R}^{10}$$

- ▶ assume 32-bit floating-point parameters (i.e., 4 bytes per parameter)
- ▶ that's 602,156,040 bytes (around 0.5GB) just to store the parameters!

Hypothetical Image Classifier

- ▶ consider using a multi-layer perceptron for image classifier



- ▶ let the image be 224-by-224 pixels
- ▶ that's $n = 224 \times 224 \times 3 = 150,528$ inputs (vectorized)
- ▶ let's have a 1000-dimensional hidden state and 10 output categories
- ▶ therefore

$$A \in \mathbb{R}^{150528 \times 1000}, \quad b \in \mathbb{R}^{1000}, \quad C \in \mathbb{R}^{1000 \times 10}, \quad d \in \mathbb{R}^{10}$$

- ▶ assume 32-bit floating-point parameters (i.e., 4 bytes per parameter)
- ▶ that's 602,156,040 bytes (around 0.5GB) just to store the parameters!
- ▶ what else is problematic? is there a better (end-to-end) approach?

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$
$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :

9	3	0	2	8	7	4	1	6	5
---	---	---	---	---	---	---	---	---	---

filter, a :

output, y :

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :										

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3									

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :		1	-2	1						
output, y :	3	5								

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :			1	-2	1					
output, y :	3	5	4							

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :				1	-2	1				
output, y :	3	5	4	-7						

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :					1	-2	1			
output, y :	3	5	4	-7	-2					

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :						1	-2	1		
output, y :	3	5	4	-7	-2	0				

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :							1	-2	1	
output, y :	3	5	4	-7	-2	0	8			

[†]In the signal processing literature a is reversed before summing.

1D Convolution

Let $x \in \mathbb{R}^n$ be an input signal and $a \in \mathbb{R}^p$ be a filter (or **kernel**). Mathematically, we write the convolution operation as[†]

$$y = a * x \quad (\in \mathbb{R}^{n-p+1})$$

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Worked Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :								1	-2	1
output, y :	3	5	4	-7	-2	0	8	-6		

[†]In the signal processing literature a is reversed before summing.

Convolution as Matrix Multiplication

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-p+1} \end{bmatrix} = \begin{bmatrix} a_1 & \dots & a_p & 0 & \dots & 0 \\ 0 & a_1 & \dots & a_p & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & a_1 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Convolution as Matrix Multiplication

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-p+1} \end{bmatrix} = \begin{bmatrix} a_1 & \dots & a_p & 0 & \dots & 0 \\ 0 & a_1 & \dots & a_p & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & a_1 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

$$y = Ax \quad (A \text{ is called a Toeplitz matrix})$$

Convolution as Matrix Multiplication

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-p+1} \end{bmatrix} = \begin{bmatrix} a_1 & \dots & a_p & 0 & \dots & 0 \\ 0 & a_1 & \dots & a_p & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & a_1 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

$$y = Ax \quad (A \text{ is called a Toeplitz matrix})$$

- we can view convolution as a linear layer with weight sharing

2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

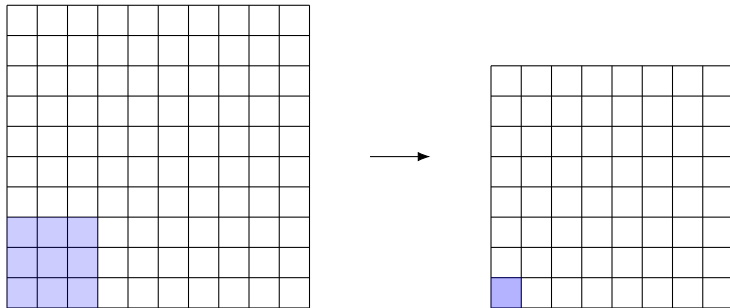
for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

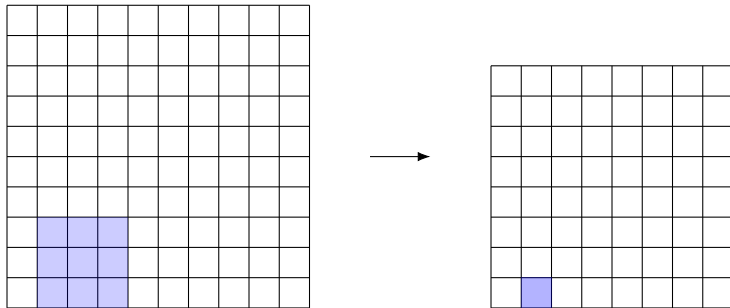


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

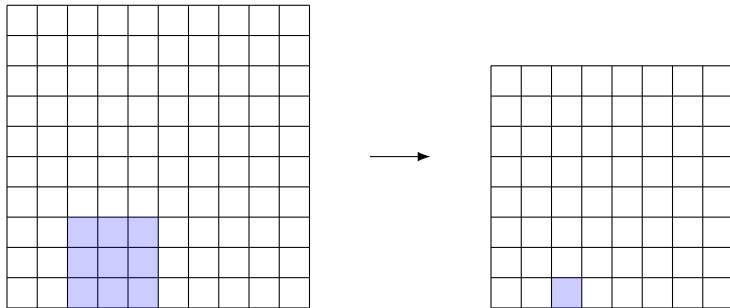


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

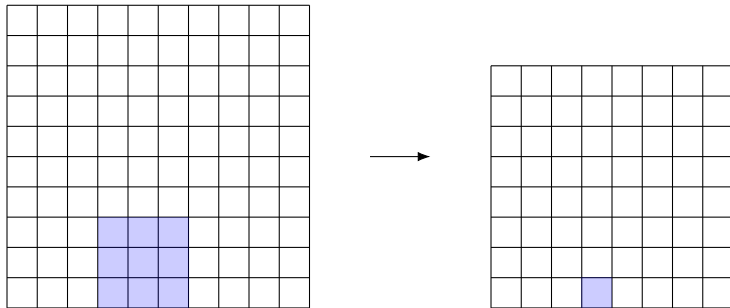


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

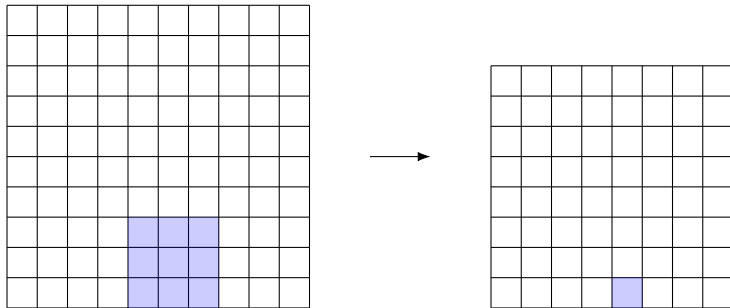


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

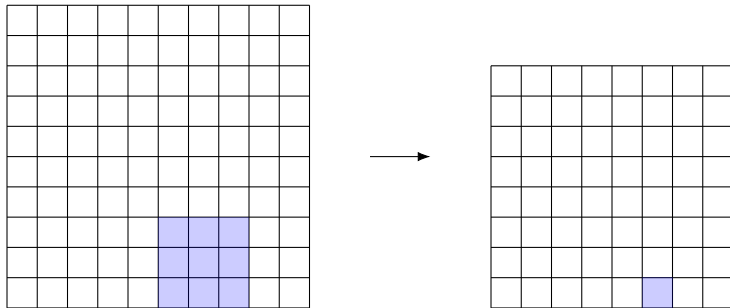


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

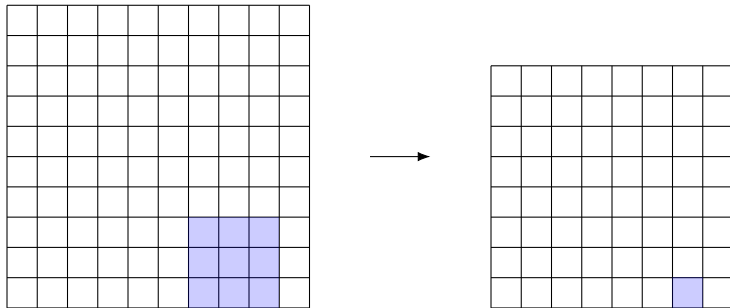


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

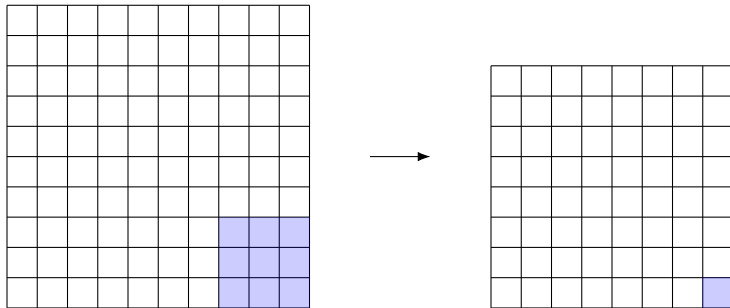


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

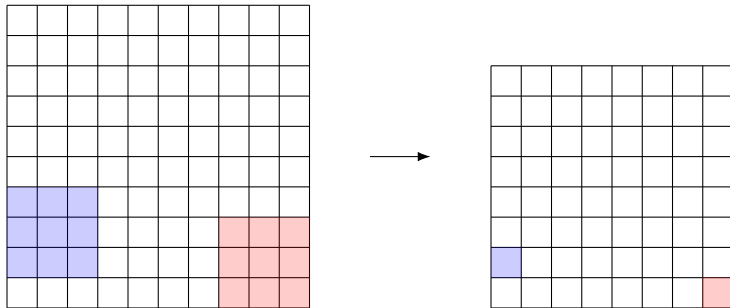


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

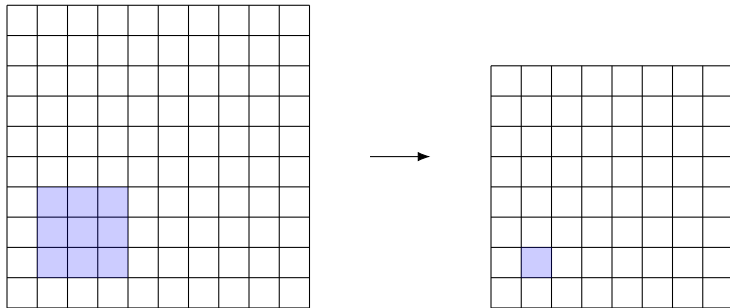


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

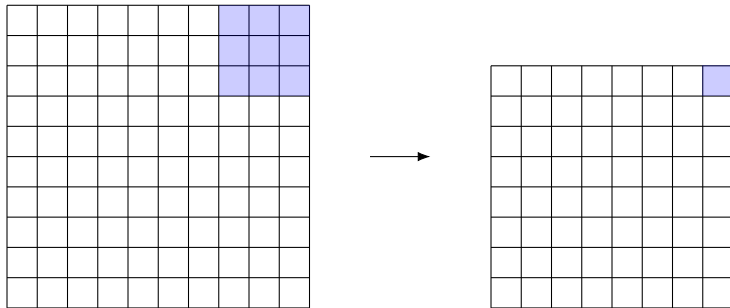


2D Convolution

Let $X \in \mathbb{R}^{n \times m}$ be an input signal and $A \in \mathbb{R}^{p \times q}$ be a filter. Then

$$Y_{ij} = \sum_{k=1}^p \sum_{\ell=1}^q A_{k,\ell} X_{i+k-1,j+\ell-1}$$

for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.



Handling Multiple Channels

Consider an image with red, green and blue channels: $X_R, X_G, X_B \in \mathbb{R}^{n \times m}$.

We can produce a feature map $Y \in \mathbb{R}^{(n-p+1) \times (m-q+1)}$ by convolving each channel with a filter A_R , A_G and A_B , and summing the output,

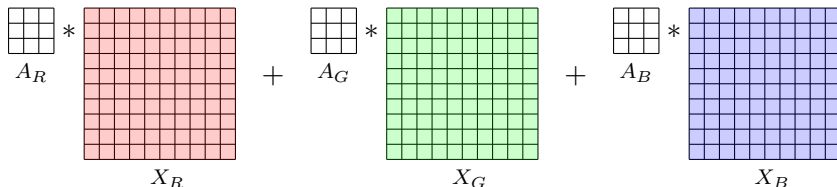
$$Y = A_R * X_R + A_G * X_G + A_B * X_B$$

Handling Multiple Channels

Consider an image with red, green and blue channels: $X_R, X_G, X_B \in \mathbb{R}^{n \times m}$.

We can produce a feature map $Y \in \mathbb{R}^{(n-p+1) \times (m-q+1)}$ by convolving each channel with a filter A_R, A_G and A_B , and summing the output,

$$Y = A_R * X_R + A_G * X_G + A_B * X_B$$



- equivalent to stacking channels into a 3-tensor and convolving with a 3D kernel

Image and Feature Map Convolutions

Let $X \in \mathbb{R}^{n \times m \times d}$ be an input signal and $A \in \mathbb{R}^{p \times q \times d}$ be a filter. Then

$$Y_{ij} = \sum_{p'=1}^p \sum_{q'=1}^q \sum_{d'=1}^d A_{p',q',d'} X_{i+p'-1,j+q'-1,d'}$$

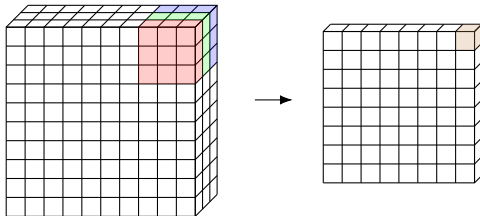
for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.

Image and Feature Map Convolutions

Let $X \in \mathbb{R}^{n \times m \times d}$ be an input signal and $A \in \mathbb{R}^{p \times q \times d}$ be a filter. Then

$$Y_{ij} = \sum_{p'=1}^p \sum_{q'=1}^q \sum_{d'=1}^d A_{p',q',d'} X_{i+p'-1,j+q'-1,d'}$$

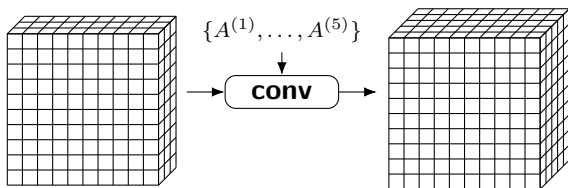
for $i = 1, \dots, n - p + 1$ and $j = 1, \dots, m - q + 1$.



- ▶ input and kernel are both 3-dimensional tensors
- ▶ third dimension equal (i.e., same number of channels), d
- ▶ confusingly, this is called Conv2d in PyTorch

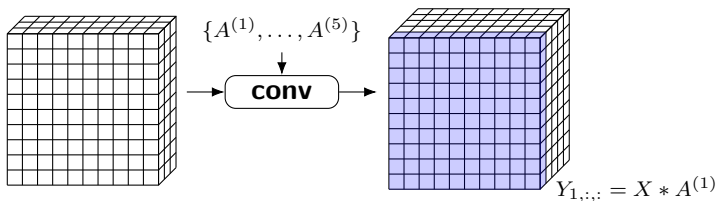
Multiple Outputs

- ▶ so far we've considered a single convolutional filter applied to an input signal
- ▶ convolutional layers in deep learning apply multiple filters and stack their output
 - ▶ $C_{\text{in}} \times H \times W$ input, X
 - ▶ K filter kernels, $\{A^{(1)}, \dots, A^{(K)}\}$
 - ▶ appropriate padding (see next slide)
 - ▶ $C_{\text{out}} \times H \times W$ output, Y , with $C_{\text{out}} = K$



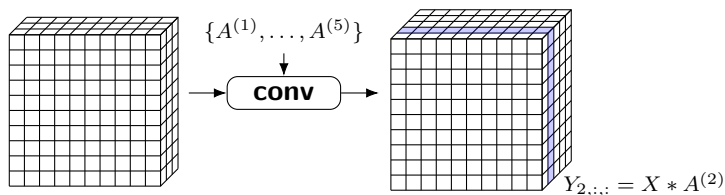
Multiple Outputs

- ▶ so far we've considered a single convolutional filter applied to an input signal
- ▶ convolutional layers in deep learning apply multiple filters and stack their output
 - ▶ $C_{\text{in}} \times H \times W$ input, X
 - ▶ K filter kernels, $\{A^{(1)}, \dots, A^{(K)}\}$
 - ▶ appropriate padding (see next slide)
 - ▶ $C_{\text{out}} \times H \times W$ output, Y , with $C_{\text{out}} = K$



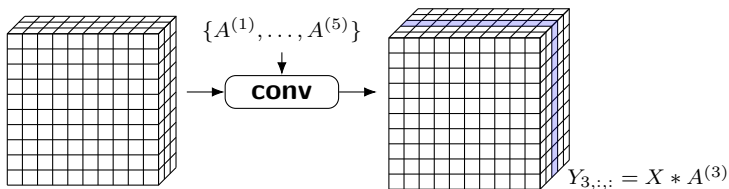
Multiple Outputs

- ▶ so far we've considered a single convolutional filter applied to an input signal
- ▶ convolutional layers in deep learning apply multiple filters and stack their output
 - ▶ $C_{\text{in}} \times H \times W$ input, X
 - ▶ K filter kernels, $\{A^{(1)}, \dots, A^{(K)}\}$
 - ▶ appropriate padding (see next slide)
 - ▶ $C_{\text{out}} \times H \times W$ output, Y , with $C_{\text{out}} = K$



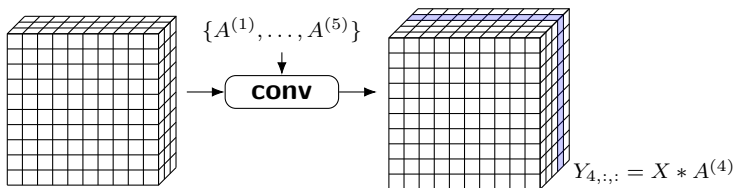
Multiple Outputs

- ▶ so far we've considered a single convolutional filter applied to an input signal
- ▶ convolutional layers in deep learning apply multiple filters and stack their output
 - ▶ $C_{\text{in}} \times H \times W$ input, X
 - ▶ K filter kernels, $\{A^{(1)}, \dots, A^{(K)}\}$
 - ▶ appropriate padding (see next slide)
 - ▶ $C_{\text{out}} \times H \times W$ output, Y , with $C_{\text{out}} = K$



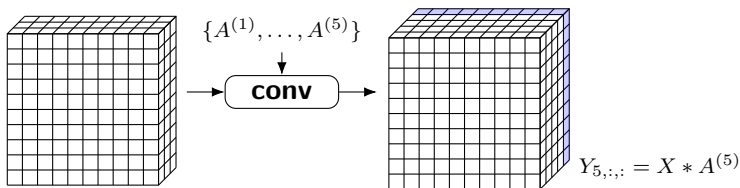
Multiple Outputs

- ▶ so far we've considered a single convolutional filter applied to an input signal
- ▶ convolutional layers in deep learning apply multiple filters and stack their output
 - ▶ $C_{\text{in}} \times H \times W$ input, X
 - ▶ K filter kernels, $\{A^{(1)}, \dots, A^{(K)}\}$
 - ▶ appropriate padding (see next slide)
 - ▶ $C_{\text{out}} \times H \times W$ output, Y , with $C_{\text{out}} = K$



Multiple Outputs

- ▶ so far we've considered a single convolutional filter applied to an input signal
- ▶ convolutional layers in deep learning apply multiple filters and stack their output
 - ▶ $C_{\text{in}} \times H \times W$ input, X
 - ▶ K filter kernels, $\{A^{(1)}, \dots, A^{(K)}\}$
 - ▶ appropriate padding (see next slide)
 - ▶ $C_{\text{out}} \times H \times W$ output, Y , with $C_{\text{out}} = K$



Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :

0	9	3	0	2	8	7	4	1	6	5	0
---	---	---	---	---	---	---	---	---	---	---	---

filter, a :

1	-2	1
---	----	---

output, y :

-15	3	5	4	-7	-2	0	8	-6	-4
-----	---	---	---	----	----	---	---	----	----

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3									

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :			1	-2	1					
output, y :	3				4					

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :					1	-2	1			
output, y :	3		4		-2					

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :							1	-2	1	
output, y :	3		4		-2		8			

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3		4		-2		8			

Dilation (atrous) Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1		-2		1					
output, y :	17									

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3		4		-2		8			

Dilation (atrous) Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :		1		-2		1				
output, y :	17	6								

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3		4		-2		8			

Dilation (atrous) Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :			1		-2		1			
output, y :	17	6	-12							

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3		4		-2		8			

Dilation (atrous) Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :				1		-2		1		
output, y :	17	6	-12	-11						

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3		4		-2		8			

Dilation (atrous) Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :					1		-2		1	
output, y :	17	6	-12	-11	6					

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3		4		-2		8			

Dilation (atrous) Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :						1		-2		1
output, y :	17	6	-12	-11	6	10				

Padding, Stride, Dilation, and Bias

(Zero) Padding Example.

input, x :	0	9	3	0	2	8	7	4	1	6	5	0
filter, a :	1	-2	1									
output, y :	-15	3	5	4	-7	-2	0	8	-6	-4		

Stride Example ($s = 2$).

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1	-2	1							
output, y :	3		4		-2		8			

Dilation (atrous) Example.

input, x :	9	3	0	2	8	7	4	1	6	5
filter, a :	1		-2		1					
output, y :	17	6	-12	-11	6	10				

Bias. Add a constant to each element so that $y = a * x + b$

Back-propagating through Convolutions

Consider again the 1D convolution,

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

Back-propagating through Convolutions

Consider again the 1D convolution,

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

- ▶ each a_j affects every y_i
- ▶ when back-propagating we need to sum these contributions

Back-propagating through Convolutions

Consider again the 1D convolution,

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

- ▶ each a_j affects every y_i
- ▶ when back-propagating we need to sum these contributions

$$\begin{aligned} \frac{dL}{da_j} &= \sum_{i=1}^{n-p+1} \frac{dL}{dy_i} \frac{dy_i}{da_j} \\ &= \sum_{i=1}^{n-p+1} \frac{dL}{dy_i} \cdot x_{i+j-1} \\ \therefore \frac{dL}{da} &= \frac{dL}{dy} * x \end{aligned}$$

Back-propagating through Convolutions

Consider again the 1D convolution,

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

- ▶ each a_j affects every y_i
- ▶ when back-propagating we need to sum these contributions

$$\begin{aligned} \frac{dL}{da_j} &= \sum_{i=1}^{n-p+1} \frac{dL}{dy_i} \frac{dy_i}{da_j} \\ &= \sum_{i=1}^{n-p+1} \frac{dL}{dy_i} \cdot x_{i+j-1} \\ \therefore \frac{dL}{da} &= \frac{dL}{dy} * x \end{aligned}$$

- ▶ likewise, each x_k affects many y_i

Back-propagating through Convolutions (2)

Consider again the 1D convolution,

$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

- ▶ each x_k affects many y_i
- ▶ however, here the resulting expression has the convolution filter reversed

Back-propagating through Convolutions (2)

Consider again the 1D convolution,

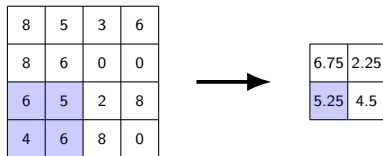
$$y_i = \sum_{j=1}^p a_j x_{i+j-1} \quad \text{for } i = 1, \dots, n - p + 1$$

- ▶ each x_k affects many y_i
- ▶ however, here the resulting expression has the convolution filter reversed

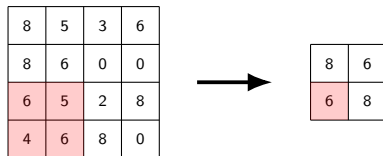
$$\begin{aligned} \frac{dL}{dx_k} &= \sum_{i=1}^{n-p+1} \frac{dL}{dy_i} \frac{dy_i}{dx_k} = \sum_{i=1}^{n-p+1} \frac{dL}{dy_i} \frac{d}{dx_k} \left(\sum_{j=1}^p a_j x_{i+j-1} \right) \\ &= \sum_{i=1}^{n-p+1} \frac{dL}{dy_i} \cdot \underbrace{a_{k-i+1}}_{\text{reversed}} \quad (\text{setting } k = i + j - 1) \\ \therefore \frac{dL}{dx} &= \frac{dL}{dy} * \mathbf{rev}(a) \end{aligned}$$

Pooling

- ▶ reduces the size of feature maps by aggregating values over a small window
- ▶ **average pooling** is the same as filtering (i.e., convolution) with a uniform kernel



- ▶ **max pooling** is non-linear; it takes the maximum value over the window



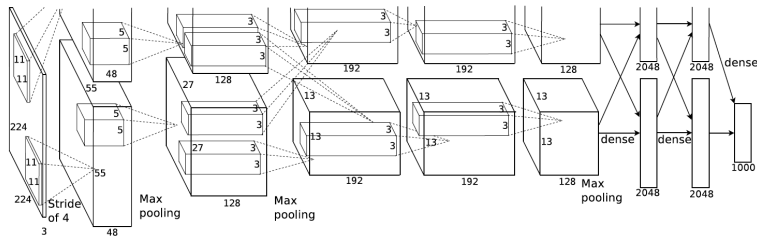
- ▶ typically the window size is 2-by-2 and the stride is 2 (i.e., window size)
- ▶ induces invariance to small shift
- ▶ the pooling operation is applied channelwise

Convolutional Neural Networks (CNNs)

- ▶ a CNN takes an image ($3 \times H \times W$ tensor) and performs successive layers of:
 - ▶ convolutions, $a * x + b$, to produce $C \times H \times W$ feature map
 - ▶ elementwise non-linear transform, e.g., ReLU
 - ▶ (feature normalization, e.g., $z = z / \|z\|_2$)
 - ▶ pooling/downsampling, e.g., $z_{st} = \max\{x_{ij} \mid i, j \in \mathcal{N}_{st}\}$
- ▶ final layers are typically “fully-connected” (i.e., an MLP) on flattened feature map

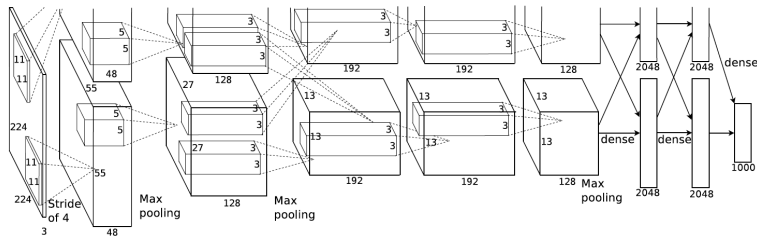
Convolutional Neural Networks (CNNs)

- ▶ a CNN takes an image ($3 \times H \times W$ tensor) and performs successive layers of:
 - ▶ convolutions, $a * x + b$, to produce $C \times H \times W$ feature map
 - ▶ elementwise non-linear transform, e.g., ReLU
 - ▶ (feature normalization, e.g., $z = z / \|z\|_2$)
 - ▶ pooling/downsampling, e.g., $z_{st} = \max\{x_{ij} \mid i, j \in \mathcal{N}_{st}\}$
- ▶ final layers are typically “fully-connected” (i.e., an MLP) on flattened feature map



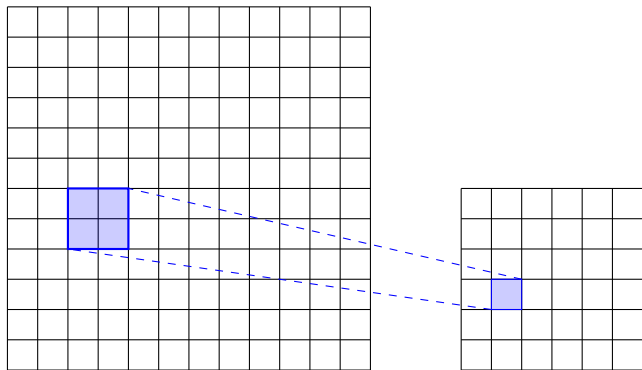
Convolutional Neural Networks (CNNs)

- ▶ a CNN takes an image ($3 \times H \times W$ tensor) and performs successive layers of:
 - ▶ convolutions, $a * x + b$, to produce $C \times H \times W$ feature map
 - ▶ elementwise non-linear transform, e.g., ReLU
 - ▶ (feature normalization, e.g., $z = z / \|z\|_2$)
 - ▶ pooling/downsampling, e.g., $z_{st} = \max\{x_{ij} \mid i, j \in \mathcal{N}_{st}\}$
- ▶ final layers are typically “fully-connected” (i.e., an MLP) on flattened feature map



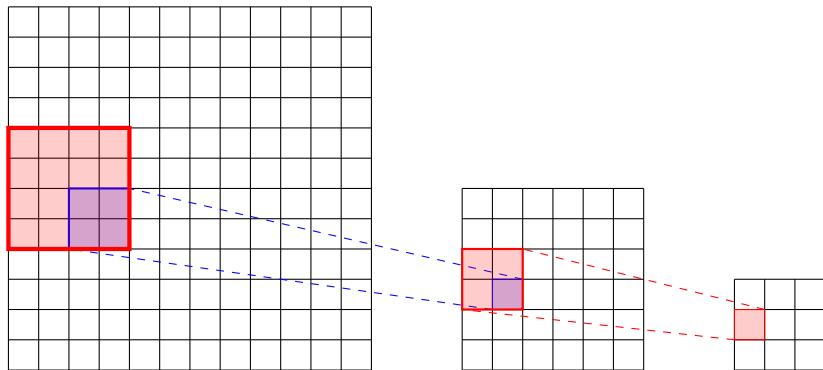
- ▶ size of a projected onto the original image is often called the “receptive field”
- ▶ many variants (see future lectures)

Receptive Field



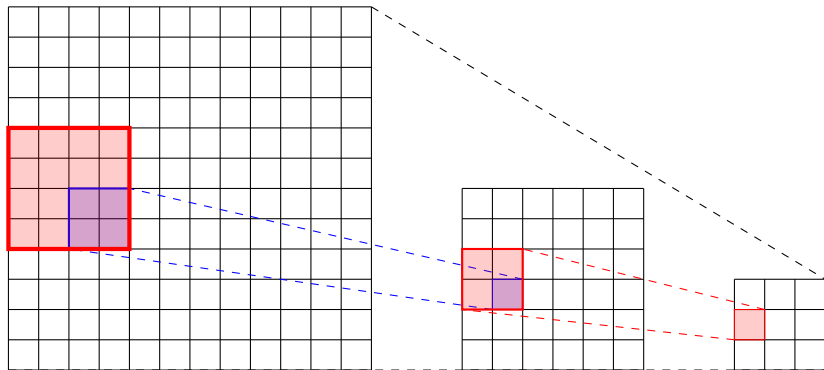
- 2-by-2 pooling (with stride of 2) between feature maps

Receptive Field



- 2-by-2 pooling (with stride of 2) between feature maps

Receptive Field



- ▶ 2-by-2 pooling (with stride of 2) between feature maps
- ▶ so 3-by-3 kernel on third layer has a receptive field of 12-by-12 pixels

Why Receptive Field Matters?

13

Why Receptive Field Matters?

12 13 14

Why Receptive Field Matters?

A

B

C

Why Receptive Field Matters?

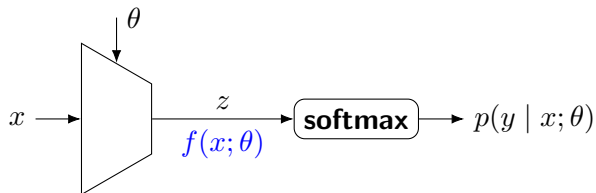
A

12 13 14

C

Image Classification

- ▶ **Task:** given an image predict it's category (from a pre-defined set)
 - ▶ more accurately, produce a distribution over categories $\implies$ softmax on final layer
- ▶ **Architecture:** CNN backbone for mapping from $\mathbb{R}^{3 \times H \times W}$ to $\mathbb{R}^K$



- ▶ **Common datasets:** MNIST, SVHN, CIFAR10/100, ImageNet, Places365, ...
- ▶ **Training:** stochastic gradient descent on cross-entropy loss
- ▶ **Evaluation:** top-1 accuracy (one minus misclassification rate), top-5 accuracy
 - ▶ more nuanced metrics such as confusion matrices and precision-vs-recall
- ▶ Test bed for new algorithms; building block for more sophisticated pipelines

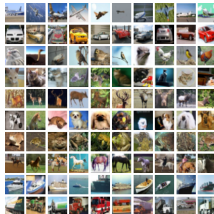
Common Image Classification Datasets



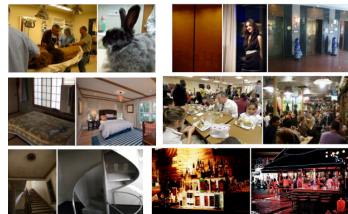
MNIST



SVHN



CIFAR10/100



Places365

evaluation metrics

Confusion Matrices

- ▶ accuracy only tells part of the story and is a poor metric for unbalanced data

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{total number of examples}}$$

- ▶ a confusion matrix can provide much more information

		predicted			
		1	2	...	K
actual	1			...	
	2			...	
	$\vdots$			$\ddots$	
	K			...	

Confusion Matrices (2)

		predicted			
		1	2	...	K
actual	1			...	
	2			...	
	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
	K			...	

- ▶ **(i,j) -th entry**: number of examples of class i that were predicted as class j
 - ▶ **row sum**: number of groundtruth examples of class i
 - ▶ **column sum**: number of examples predicted as class j
 - ▶ **diagonal sum**: number of correctly classified examples
 - ▶ **total sum**: total number of examples
-
- ▶ accuracy is (diagonal sum) / (total sum)
 - ▶ number of rows does not need to equal number of columns
 - ▶ allows fine-grained or coarse-grained analysis
 - ▶ cumbersome for large label spaces

Accuracy: Micro vs Macro Averaging

- ▶ overall accuracy is an example of **micro-averaging**

$$\text{accuracy}_{\text{micro}} = \frac{\text{number of correct predictions}}{\text{total number of examples}}$$

Accuracy: Micro vs Macro Averaging

- ▶ overall accuracy is an example of **micro-averaging**

$$\text{accuracy}_{\text{micro}} = \frac{\text{number of correct predictions}}{\text{total number of examples}}$$

- ▶ for unbalanced data we might wish to treat each class equally
- ▶ this is an example of **macro-averaging**

$$\text{accuracy}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{\text{number of correct predictions for class } k}{\text{total number of examples for class } k}$$

Accuracy: Micro vs Macro Averaging

- ▶ overall accuracy is an example of **micro-averaging**

$$\text{accuracy}_{\text{micro}} = \frac{\text{number of correct predictions}}{\text{total number of examples}}$$

- ▶ for unbalanced data we might wish to treat each class equally
- ▶ this is an example of **macro-averaging**

$$\text{accuracy}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{\text{number of correct predictions for class } k}{\text{total number of examples for class } k}$$

- ▶ more generally, we can compute weighted accuracy

Precision and Recall for Binary Classification

		predicted	
		0	1
actual	0	TN	FP
	1	FN	TP

- ▶ **TP**: true positive, hit, detection
- ▶ **TN**: true negative, correct rejection
- ▶ **FP**: false positive, false alarm, Type I error
- ▶ **FN**: false negative, miss, Type II error
- ▶ depends on detection threshold, i.e., $p(y \mid x; \theta) \geq t$

Precision and Recall for Binary Classification

		predicted	
		0	1
actual	0	TN	FP
	1	FN	TP

- ▶ **TP**: true positive, hit, detection
- ▶ **TN**: true negative, correct rejection
- ▶ **FP**: false positive, false alarm, Type I error
- ▶ **FN**: false negative, miss, Type II error
- ▶ depends on detection threshold, i.e., $p(y | x; \theta) \geq t$

Derived Statistics

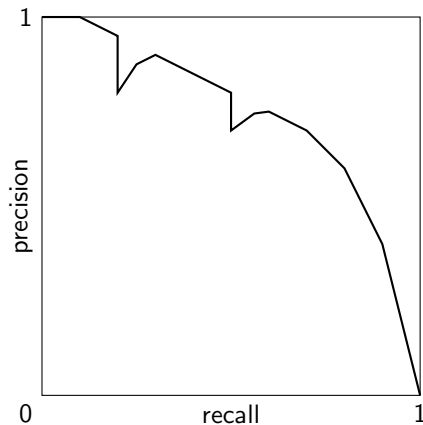
- ▶ **recall**, true positive rate, sensitivity, hit rate: $TP / (TP + FN)$
- ▶ **precision**, positive predictive power: $TP / (TP + FP)$
- ▶ true negative rate, specificity: $TN / (TN + FP)$
- ▶ **accuracy**: $(TP + TN) / (TN + FP + FN + TP)$
- ▶ **F₁-score**: $2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$
- ▶ false alarm rate, false positive rate, fall-out; false discovery rate; F_β -score, ...

Precision-Recall (PR) Curves

- ▶ classification rule,

$$p(y = 1 \mid x; \theta) \geq t$$

- ▶ precision-recall curve generated by sweeping over threshold t
 - ▶ decreasing from left-to-right



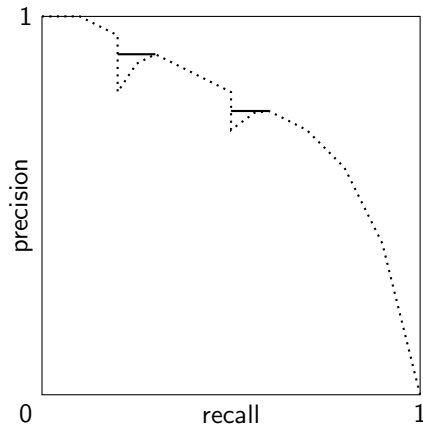
Precision-Recall (PR) Curves

- ▶ classification rule,

$$p(y = 1 \mid x; \theta) \geq t$$

- ▶ precision-recall curve generated by sweeping over threshold t
 - ▶ decreasing from left-to-right

$$\text{prec}_{\text{interp}}(r) = \max_{r' \geq r} \{\text{prec}(r')\}$$

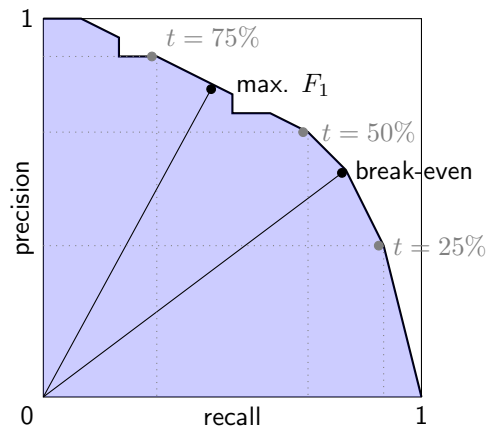


Precision-Recall (PR) Curves

- ▶ classification rule,

$$p(y = 1 \mid x; \theta) \geq t$$

- ▶ area under the curve is called **average precision (AP)**
 - ▶ traditionally computed on 11-point interpolated curve

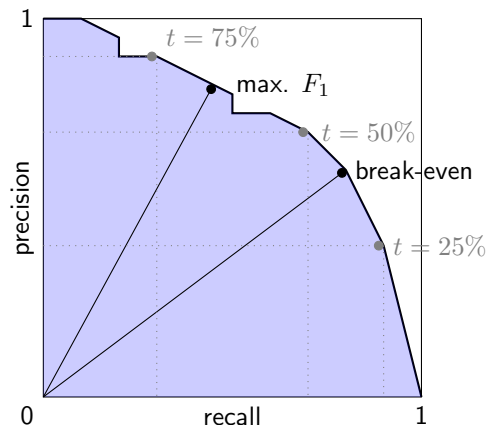


Precision-Recall (PR) Curves

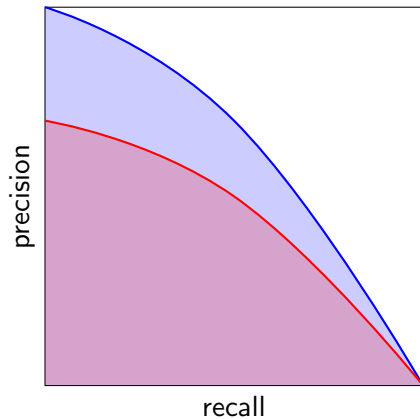
- ▶ classification rule for class k ,

$$p(y = k \mid x; \theta) \geq t$$

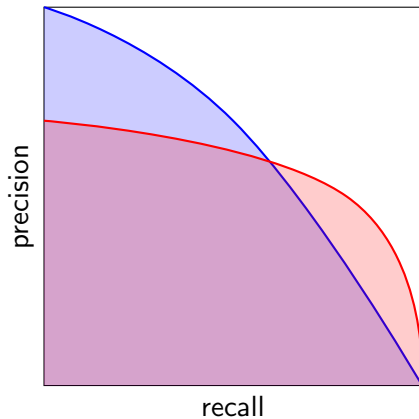
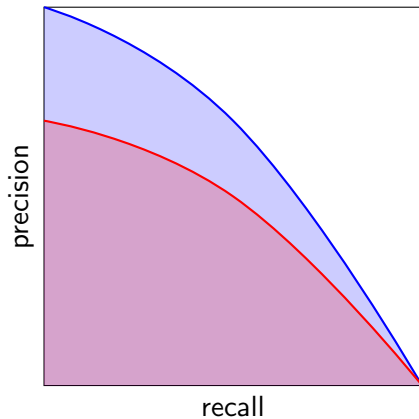
- ▶ area under the curve is called **average precision (AP)**
 - ▶ traditionally computed on 11-point interpolated curve
- ▶ for multiclass tasks we aggregate AP on each category to get **mean average precision (mAP)**



Comparing Models



Comparing Models



Other Metrics and Considerations

Other metrics you will often see:

- ▶ number of model parameters
- ▶ speed of inference

Other Metrics and Considerations

Other metrics you will often see:

- ▶ number of model parameters
- ▶ speed of inference

Other considerations in reporting performance and comparing models:

- ▶ run with multiple different random seeds and report mean and standard deviation
- ▶ report trends rather than single operating points
- ▶ these are not always feasible with large models/datasets

Other Metrics and Considerations

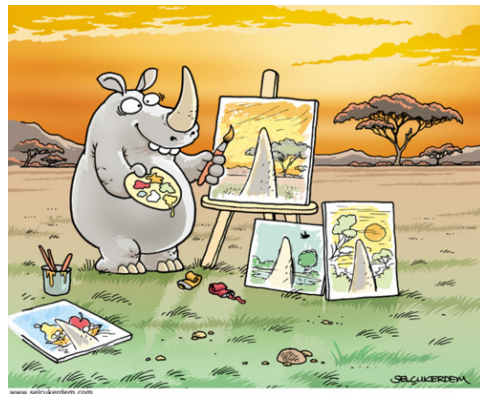
Other metrics you will often see:

- ▶ number of model parameters
- ▶ speed of inference

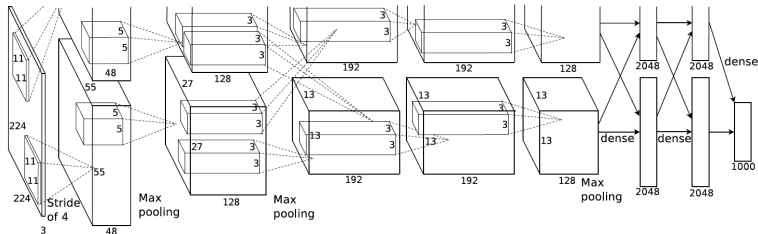
Other considerations in reporting performance and comparing models:

- ▶ run with multiple different random seeds and report mean and standard deviation
- ▶ report trends rather than single operating points
- ▶ these are not always feasible with large models/datasets

- ▶ **always** visualize your data and results to make sure that your metrics align with your intuition

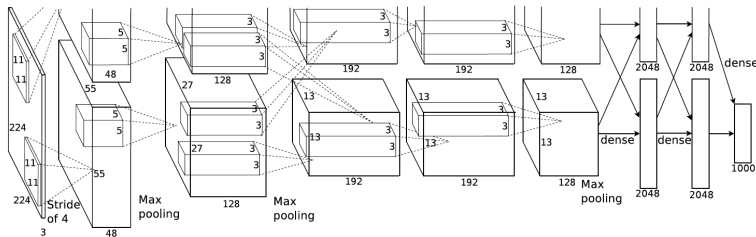


Why Convolutions?



$$\begin{aligned}
 \# \text{ parameters} &= 2 \times (11 \times 11 \times 3 + 1) \times 48 + && (\text{conv1}) \\
 &2 \times (5 \times 5 \times 48 + 1) \times 128 + && (\text{conv2}) \\
 &2 \times (3 \times 3 \times 128 + 1) \times 192 + && (\text{conv3}) \\
 &2 \times (3 \times 3 \times 192 + 1) \times 192 + && (\text{conv4}) \\
 &2 \times (3 \times 3 \times 192 + 1) \times 128 + && (\text{conv5}) \\
 &(6 \times 6 \times (128 + 128) + 1) \times (2048 + 2048) + && (\text{fc1}) \\
 &(2048 + 2048 + 1) \times (2048 + 2048) + && (\text{fc2}) \\
 &(2048 + 2048 + 1) \times 1000 && (\text{fc3}) \\
 &= 60,522,856
 \end{aligned}$$

Why Convolutions?



parameters = 60,522,856

(cf. 150,539,010 for MLP from beginning of lecture)

- ▶ massive reduction in parameter count for much deeper network
- ▶ invariant to small pixel shifts — better generalization
- ▶ parameter sharing helps to generalize

other architectural and training ingredients

Data Augmentation



- ▶ data augmentation generates new training data from existing data
- ▶ helps improve generalization (i.e., prevent overfitting)
- ▶ transformations are applied to the data specific for the task

Data Augmentation



- ▶ data augmentation generates new training data from existing data
- ▶ helps improve generalization (i.e., prevent overfitting)
- ▶ transformations are applied to the data specific for the task

geometric

- ▶ rotation, flipping, cropping



colour space

- ▶ brightness, contrast, saturation, jittering



noise

- ▶ gaussian, salt & pepper, random erase



Batch Normalization

(Ioffe and Szegedy, 2015)

- ▶ recall from last week that normalizing data helps convergence during training
- ▶ **batch normalization** (BN) is a technique for on-the-fly centering and scaling feature maps within the network
- ▶ consider a batch $\mathcal{B} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ with intermediate features $z^{(i)} \in \mathbb{R}$

$$\mu_{\mathcal{B}} = \frac{1}{N} \sum_{i=1}^N z^{(i)} \quad \text{and} \quad \sigma_{\mathcal{B}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (z^{(i)} - \mu_{\mathcal{B}})^2}$$
$$\hat{z}^{(i)} = \gamma \left(\overbrace{\frac{z^{(i)} - \mu_{\mathcal{B}}}{\sigma_{\mathcal{B}}}}^{\text{normalization}} \right) + \beta$$

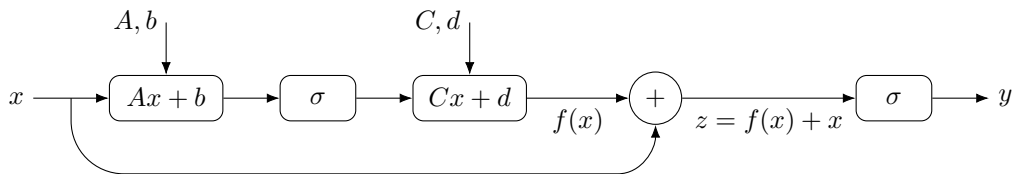
where γ and β are learnable parameters

- ▶ running averages of μ and σ are maintained for use at test time

ResNet

(He et al., 2016)

- ▶ one way to address the issue of diminishing gradients is via short-cut connections
- ▶ the ResNet block is a classic example



- ▶ the gradient through the ResNet block is

$$\frac{dL}{dx} = \frac{dL}{dz} \left(\frac{dz}{dx} + 1 \right)$$

- ▶ ResNet also models “residual signals”, which may help lead to it’s improved empirically performance and de facto standard backbone for many problems